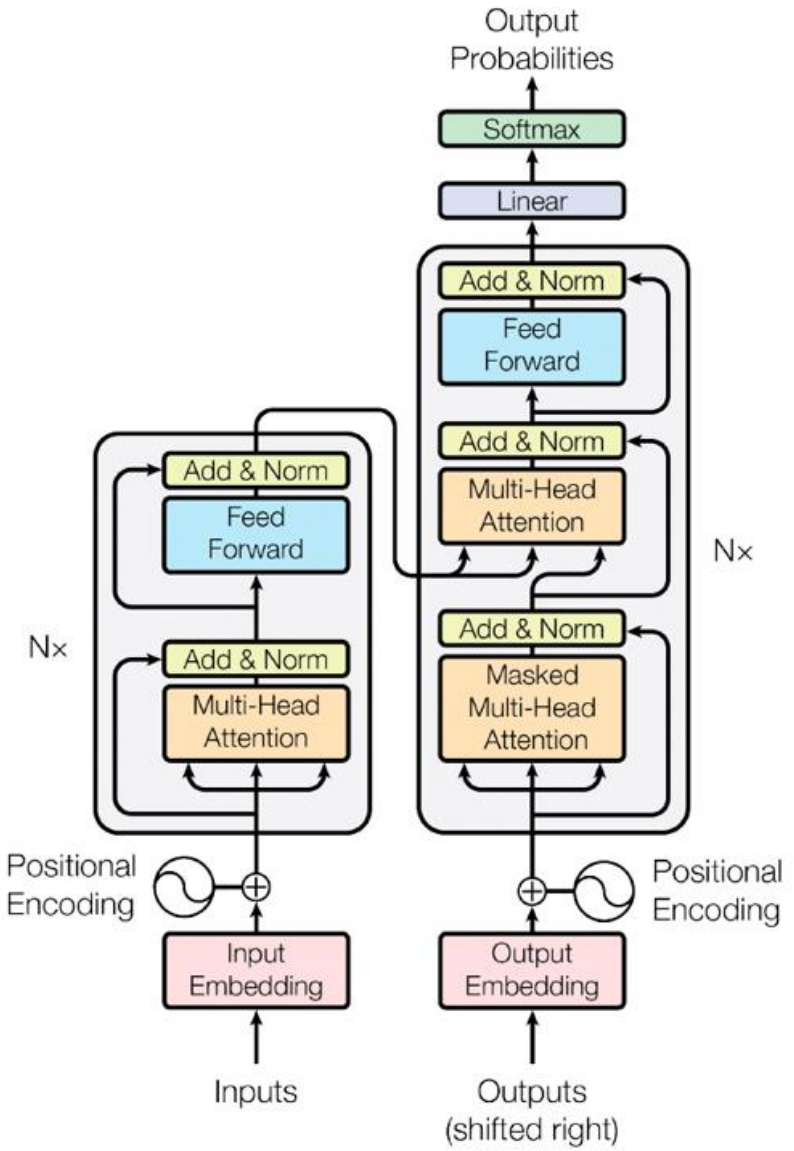
Transformers

The Architecture Behind Modern AI, ChatGPT, and Large Language Models



The Core Problem

- The **bank** of the river was full of mud.
- The **bank** approved my loan.

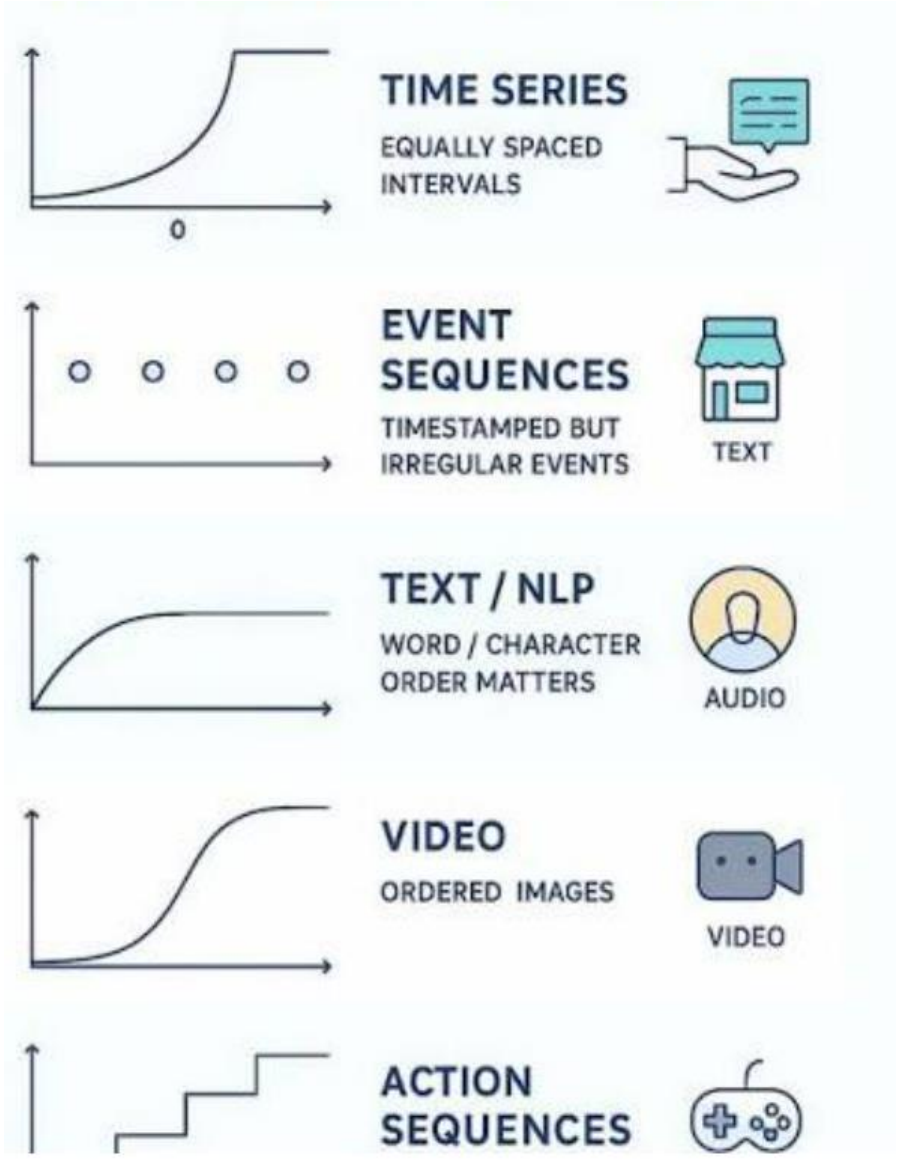
- The Ideal Goal: We need the represent of "**bank**" to dynamically shift based on the words surrounding it.

- The main challenge: how do we let every word talk to every other word in a single parallel step?

- Solution: **Self Attention Mechanism**

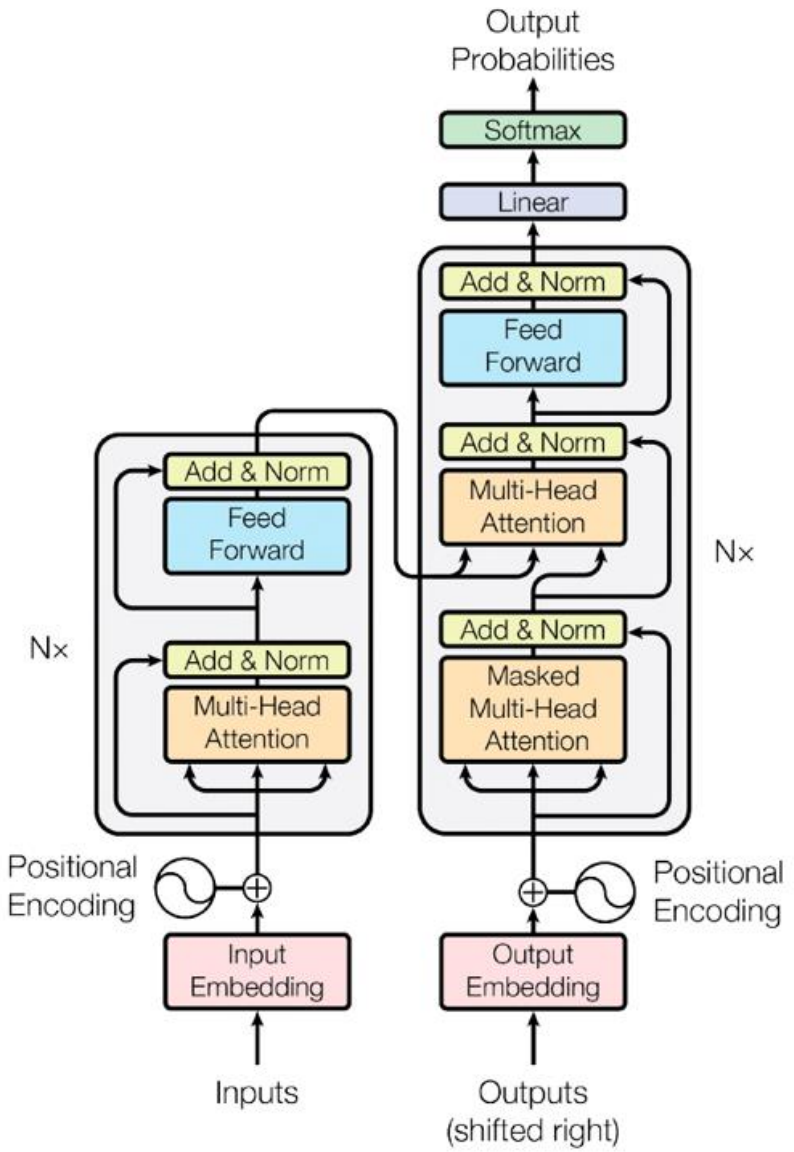
Sequential and Non-sequential Data

Aspect	Sequential Data	Non-sequential data
Order of data	Order of data matters.	Order of data does not matter.
Dependency	Current data depends on previous data for context and prediction.	Each data point is independent ; no dependency on order of other features.
Models used	Models like RNN, LSTM, Transformers handle sequential data. 1D CNN is also used for local patterns in sequences like text and time series.	Models like ANN, 2D CNN, Decision Tree, Logistic Regression, Random Forest which are used for non sequential data.
Use case	Language translation, stock price prediction, speech recognition, text classification.	Student placement prediction, customer churn, image classification tasks.



Transformers

- It is introduced in 201 by Ashish Vaswani.
- It is originally designed for sequence to-sequence task.
- Seq-2-seq Task
  - Text generation,
  - Text summarization,
  - Question-answering
  - Machine Translations



Attention Is All You Need

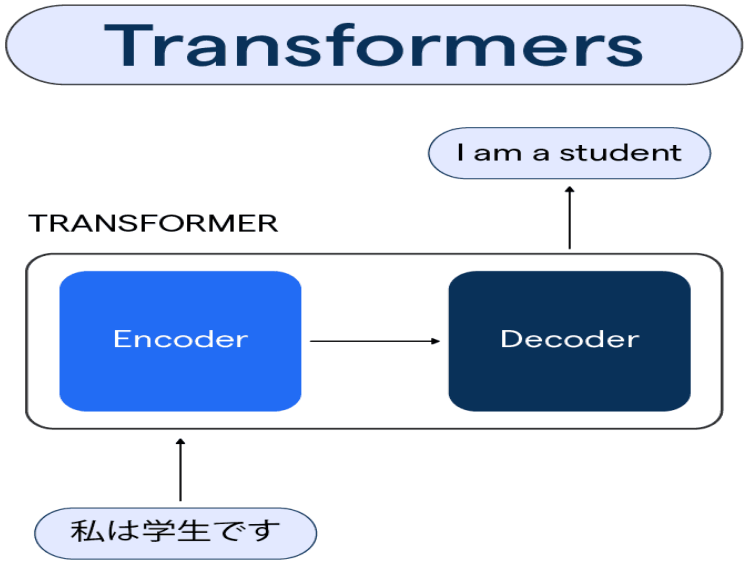
Ashish Vaswani* Google Brain avaswani@google.com
Noam Shazeer* Google Brain noam@google.com
Niki Parmar* Google Research nikip@google.com
Jakob Uszkoreit* Google Research usz@google.com
Llion Jones* Google Research llion@google.com
Aidan N. Gomez*† University of Toronto aidan@cs.toronto.edu
Łukasz Kaiser* Google Brain lukaszkaiser@google.com
Illia Polosukhin* illia.polosukhin@gmail.com

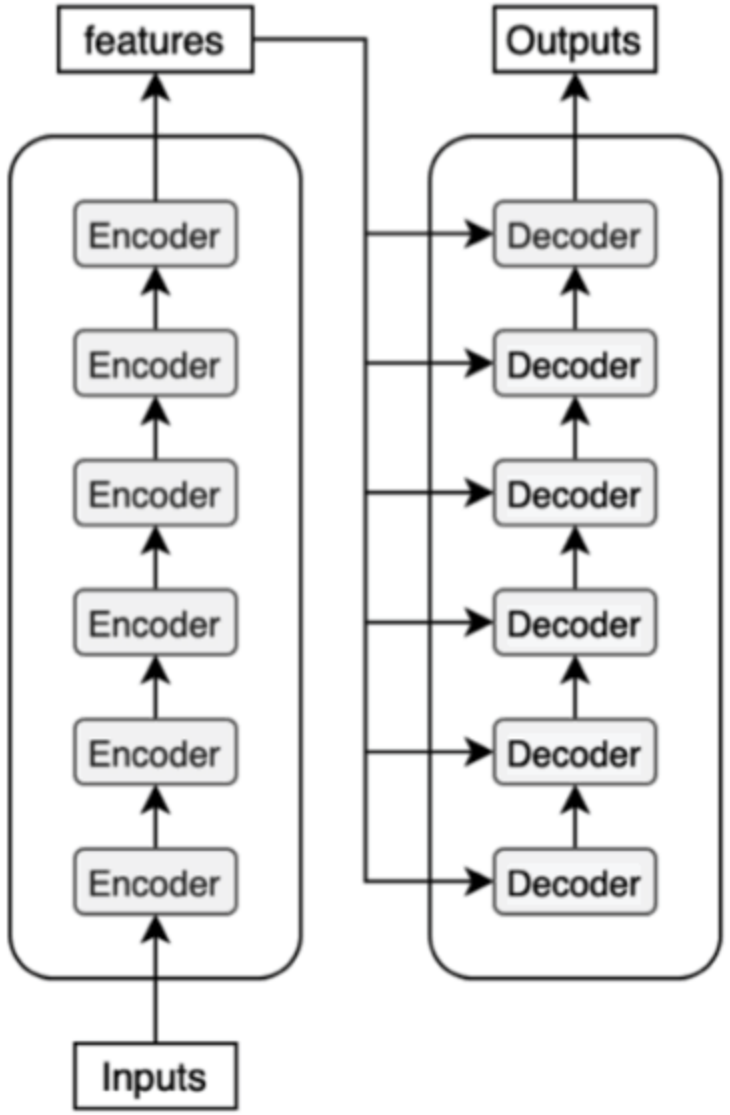
Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks in an encoder-decoder configuration. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.0 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

1706.03762 v1

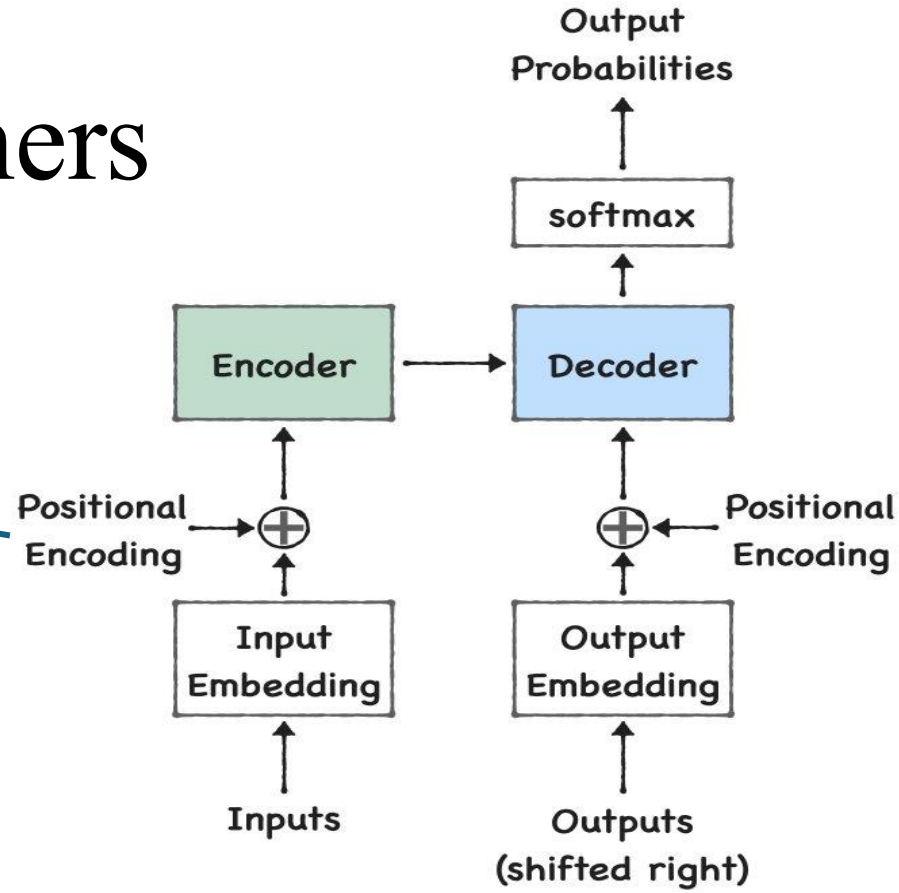
Closer Inside of Transformers

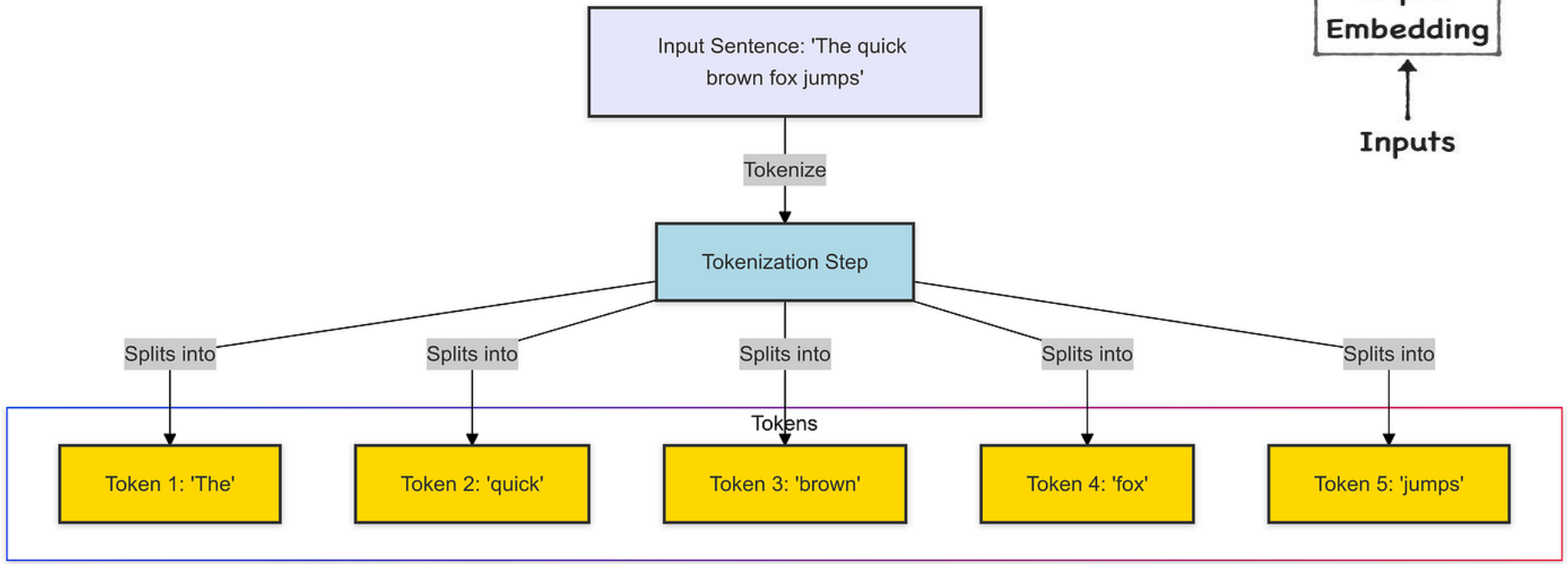




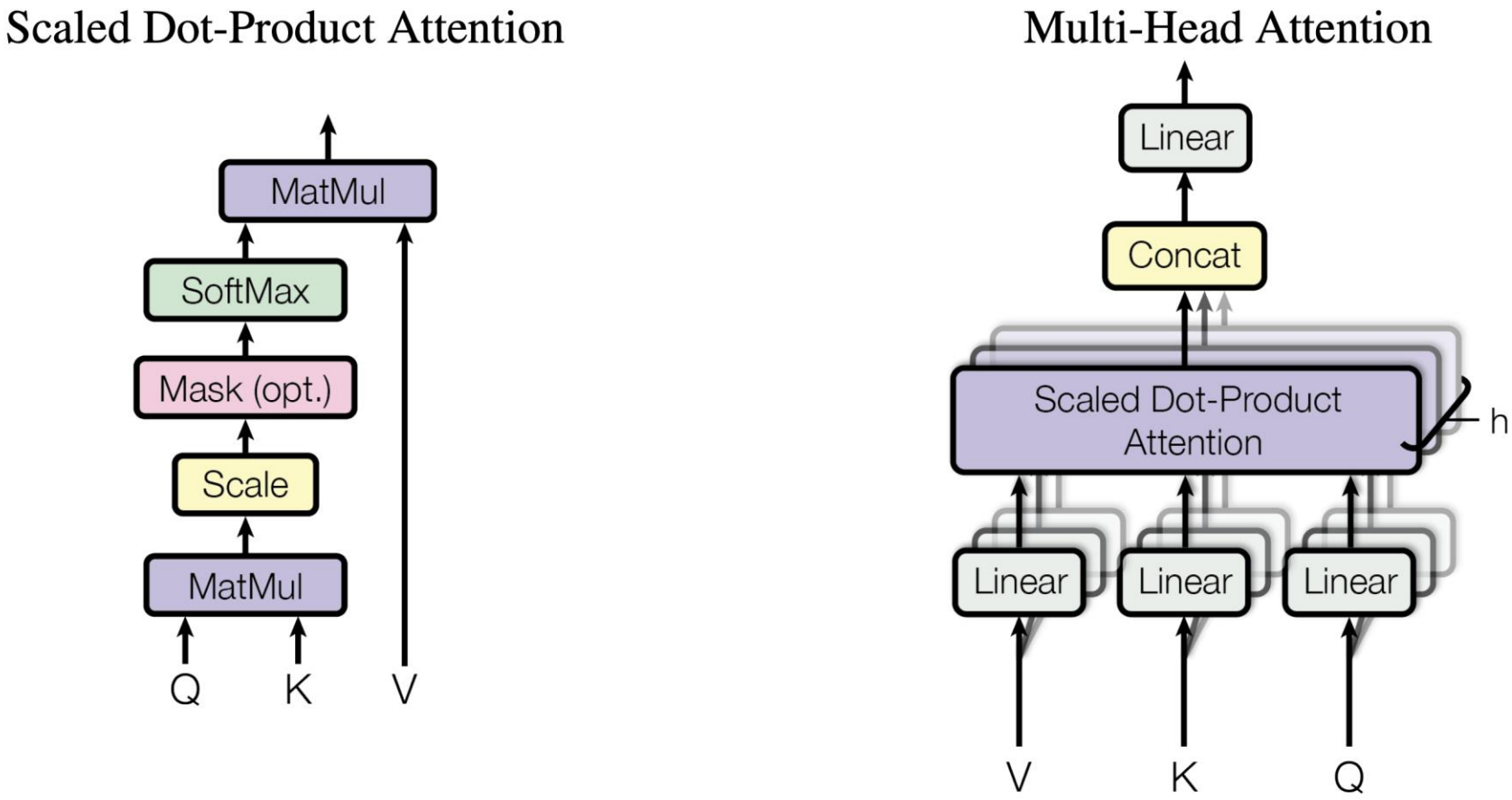
Closer Inside of Transformers

$$PE_{(pos,i)} = \begin{cases} \sin\left(\frac{pos}{10000^{i/d_{\text{model}}}}\right) & \text{if } i \bmod 2 = 0 \\ \cos\left(\frac{pos}{10000^{(i-1)/d_{\text{model}}}}\right) & \text{otherwise} \end{cases}$$

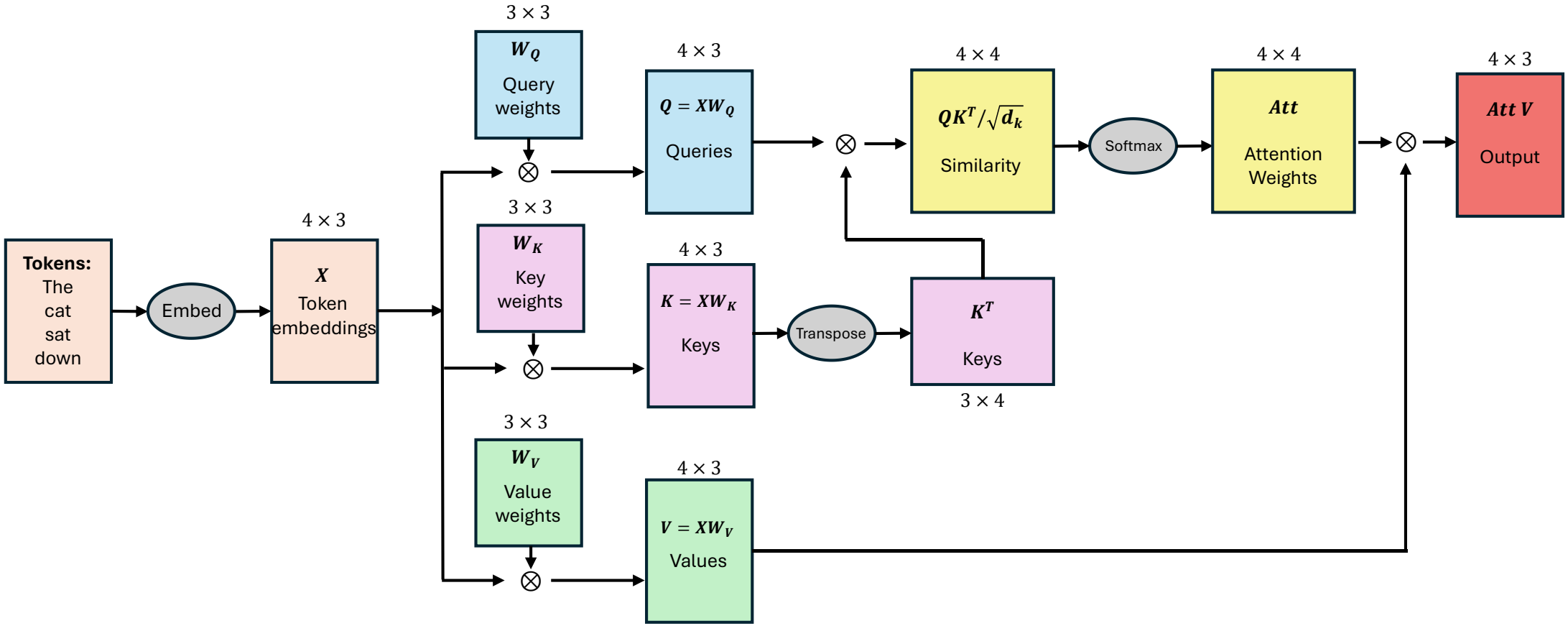




Closer Inside of Transformers



Self Attention: Computational Flow



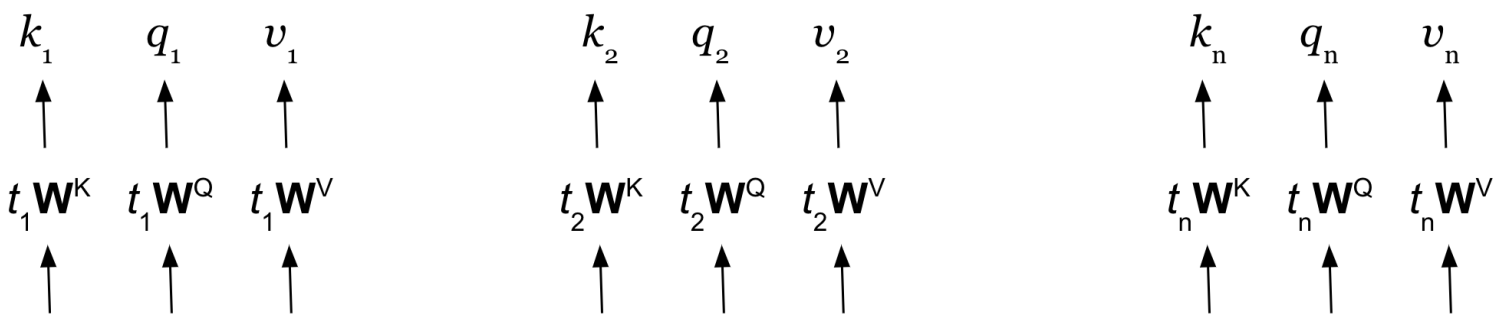
Step-by-step explanation of Self-Attention

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

to compute Attention for all input tokens at once use Matrix operations (same as)
where, Q = XW^Q, K = XW^K, and V = XW^V

$$Attention_{token\_n} = \sum w_i v_i$$

$$w_1 = k_1 \bullet q_n \qquad w_2 = k_2 \bullet q_n \qquad w_n = k_n \bullet q_n$$



Embedding for Token *1* (t_1)
vector of dim_{embed}

Embedding for Token *2* (t_2)
vector of dim_{embed}

Embedding for Token *n* (t_n)
vector of dim_{embed}

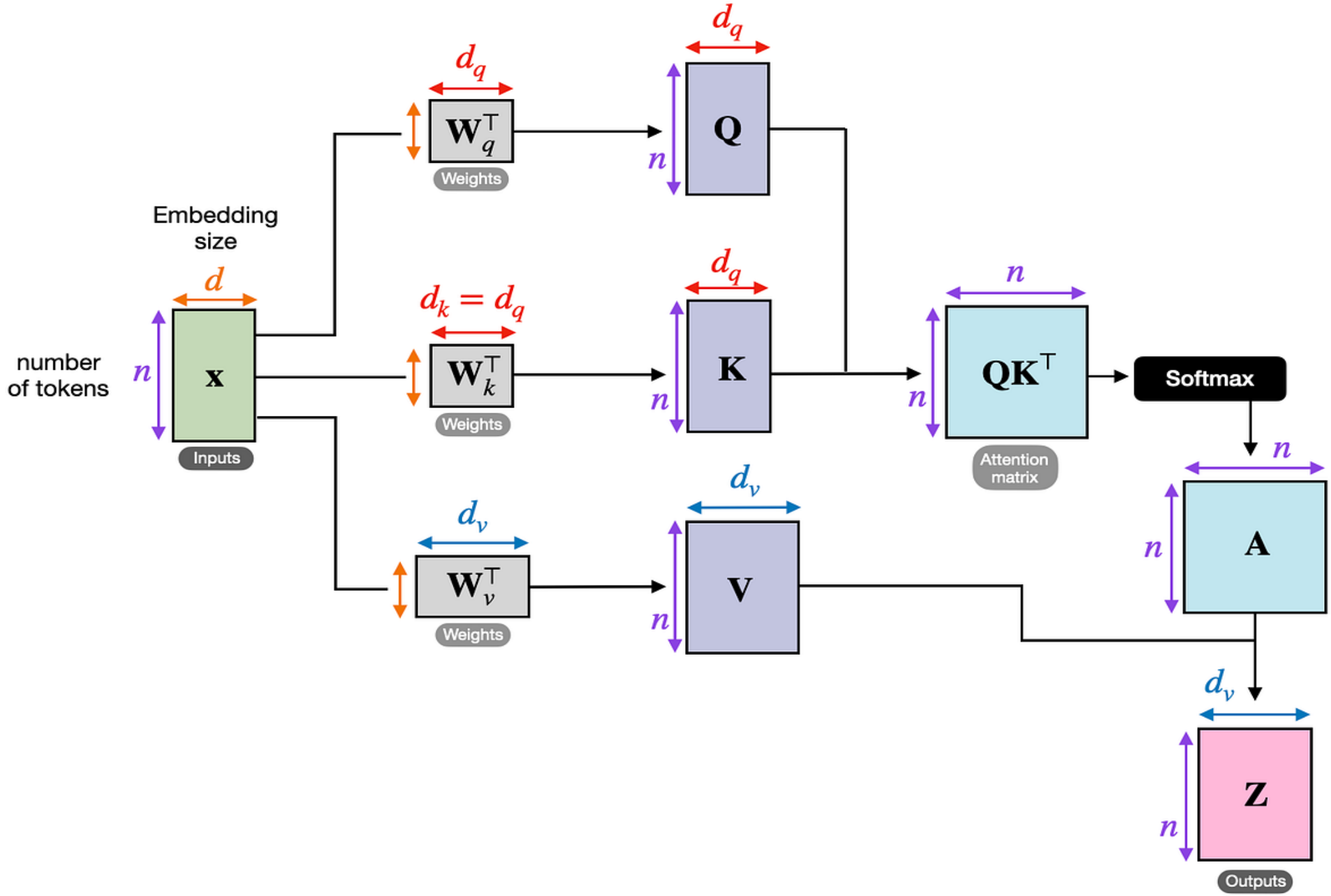
④ Attention for token *n* is computed as a weighted sum of values of all the input tokens

③ dot product of key vector (k_i) of every input token with the query vector of n^{th} token (q_n) is used to dynamically compute weight (w_i) of each input token

② For each head, we initialize three matrices of *dim(embed, head_size)* called Key ($\mathbf{W}^K$), Query ($\mathbf{W}^Q$), and Value ($\mathbf{W}^V$)

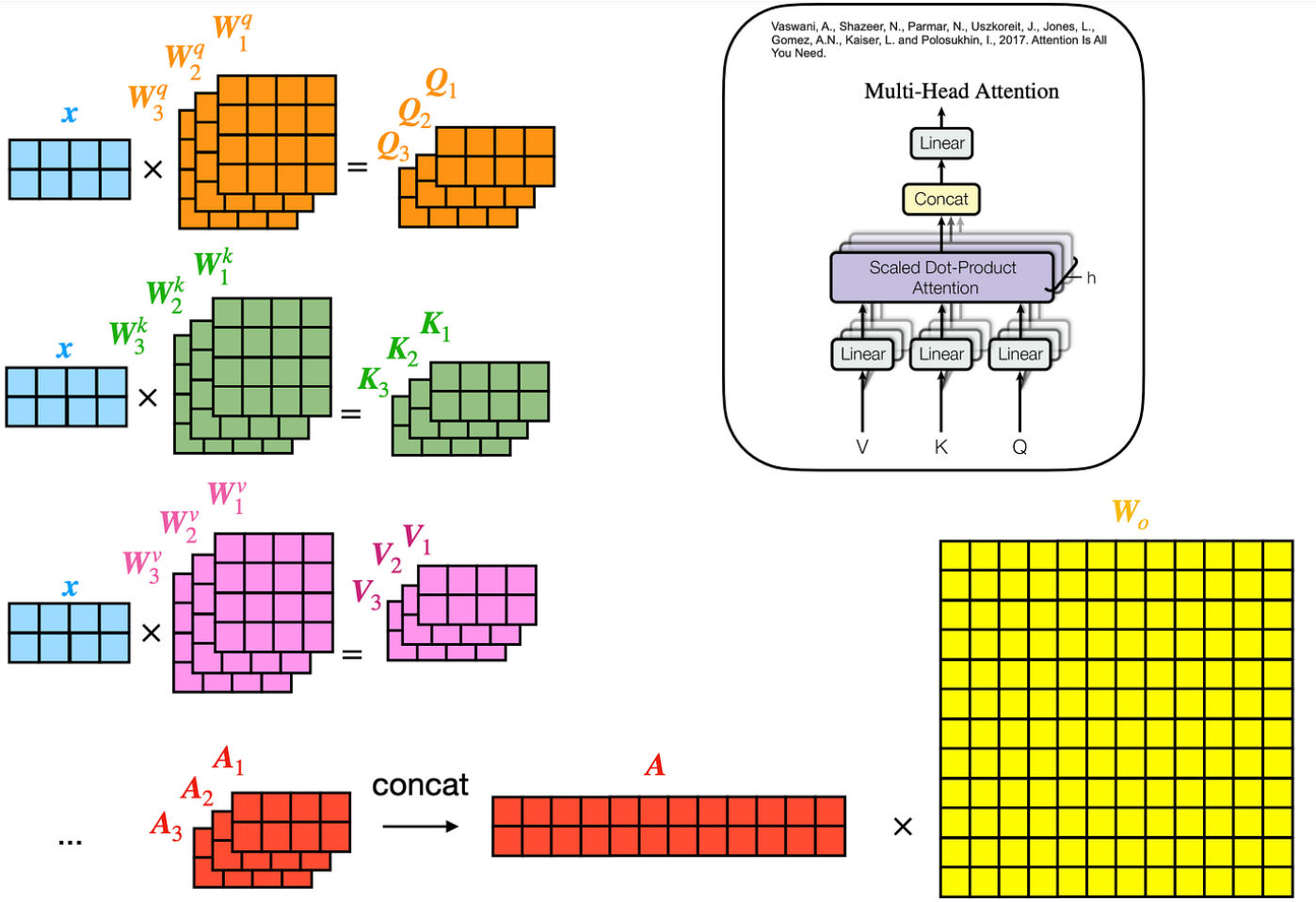
Embedding for each of the input token is multiplied by $\mathbf{W}^K$, $\mathbf{W}^Q$, and $\mathbf{W}^V$ matrices to generate k, q, and v vectors

① Input is represented as matrix $\mathbf{X}$ of *dim(n, embed)*, where *n* is the num of input tokens


Embedding size
number of tokens
d
n
x
Inputs
d_q
$\mathbf{W}_q^\top$
Weights
$d_k = d_q$
$\mathbf{W}_k^\top$
Weights
d_v
$\mathbf{W}_v^\top$
Weights
d_q
Q
n
d_q
K
n
d_v
V
n
n
$\mathbf{QK}^\top$
n
Attention matrix
Softmax
n
A
n
d_v
Z
n
Outputs

https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/tutorial6/Transformers_and_MHAttention.html

Multi-head Attention



https://poloclub.github.io/transformer-explainer/

Some Important Points

- Transformers are designed to be very deep. Some models contain more than 24 blocks in the encoder. Hence, the residual connections are crucial for enabling a smooth gradient flow through the model.
- Without the residual connection, the information about the original sequence is lost.
- The Layer Normalization also plays an important role in the Transformer architecture as it enables faster training and provides small regularization. Additionally, it ensures that the features are in a similar magnitude among the elements in the sequence.
- Additionally to the Multi-Head Attention, a small fully connected feed-forward network is added to the model, which is applied to each position separately and identically. Specifically, the model uses a LinearReLULinear MLP. The full transformation including the residual connection can be expressed as:

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$
$$x = \mathrm{LayerNorm}(x + \mathrm{FFN}(x))$$

Three Core Transformer Architectures

Encoder-Only, Decoder-Only, and Encoder-Decoder

Encoder-Only

(Autoencoding)

- Processes entire input
- Masked language modeling
- BERT

Decoder-Only

(Autoregressive)

- Causal (one-way) processing
- Next token prediction
- GPT

Encoder-Decoder

(Seq2Seq)

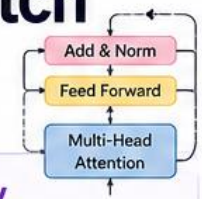
- Encodes input
- Autoregressive decoding
- T5, BART

Comparing Transformer-based Models

Aspect	T5	BERT	GPT-2
Architecture	Encoder-Decoder	Encoder-Only	Decoder-Only
Primary Use Case	Sequence-to-sequence tasks	Text understanding tasks	Text generation tasks
Pre-training Objective	Denoising span corruption	Masked Language Modeling	Causal Language Modeling
Position Embedding	Relative position embeddings	Absolute position embeddings	Absolute position embeddings
Parameter Count	Variable (e.g., T5-base: 220M, T5-Large: 770M)	Variable (e.g., BERT-Base: 110M, BERT-Large: 340M)	Variable (e.g., GPT-2 Medium: 345M, GPT-2 Large: 774M)
Strengths	Adaptability to diverse tasks with a single architecture.	Strong text understanding from bidirectional context	Fluent text generation with long-term coherence
Weakness	Requires both encoder and decoder during inference, making it more resource-intensive	Is limited to text understanding, cannot generate coherent text	Has limited understanding of bidirectional context
Example Models	T5-Small, T5-Large, T5-11B	BERT-Base, BERT-Large, RoBERTa	GPT-2, GPT-3, ChatGPT

Challenges of Developing a Transformer Model from Scratch

Building a Transformer model from scratch is a complex task that involves challenges across data, computation, architecture, training, and deployment.



1 Massive Data Requirement

Transformers learn patterns from huge amounts of data.

Challenges

- Collecting high-quality datasets
- Removing duplicates and noise
- Handling biased or toxic content
- Data labeling (for supervised tasks)

Example

A small dataset of 10,000 sentences may cause the model to memorize rather than generalize.

Modern LLMs are trained on:

Books · Websites · Research papers · Code repositories · Conversations

Often totaling trillions of tokens.

2 High Computational Cost

Training Transformers requires enormous computing power.

Challenges

- Multiple GPUs/TPUs needed
- High electricity consumption
- Long training times
- Expensive hardware

Example

Model	Parameters
BERT Base	110M
GPT-2	1.5B
Llama 3 8B	8B
GPT-class models	Hundreds of billions+

Training large models may require:

Multi-GPU clusters · Distributed training · High-speed networking

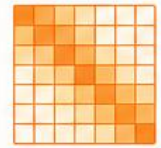
3 Memory Consumption

Self-attention stores large matrices.

Challenge

Attention complexity: $O(n^2)$

where: n = sequence length



Example

Tokens	Attention Matrix Size	Growth
100	10,000	100^2
1,000	1,000,000	$1{,}000^2$
10,000	100,000,000	$10{,}000^2$

Long documents can quickly exhaust GPU memory.

7 Training Stability

Large Transformers are difficult to train.

Common Problems

Exploding Gradients
Weights become extremely large.

Vanishing Gradients
Weights stop updating.

Loss Instability
Training loss fluctuates wildly.

Solutions

Layer Normalization · Residual Connections · Gradient Clipping · Learning Rate Scheduling

8 Long Training Time

Training may take:

Model Size	Training Time
Small Transformer	Hours
Medium Model	Days
Large LLM	Weeks
Foundation Models	Months

Challenges

Hardware failures

Interrupted training

Checkpoint management

11 Overfitting

The model memorizes training data.

Symptoms

- Excellent training performance
- Poor real-world performance

Solutions

More data · Dropout · Regularization (L1/L2) · Data augmentation

13 Model Interpretability

Transformers are often considered "black boxes."

Challenge

Difficult to answer: Why did the model make this prediction?

Researchers use:

Attention visualization · Feature attribution · Explainable AI techniques

Other Key Challenges (Bonus)

Complex Architecture Design
Choosing hyperparameters, layers, heads, dimensions, etc.

Data Quality & Bias
Biased or noisy data leads to unfair or harmful outputs.

Deployment & Inference
Optimizing for latency, throughput and resource constraints.

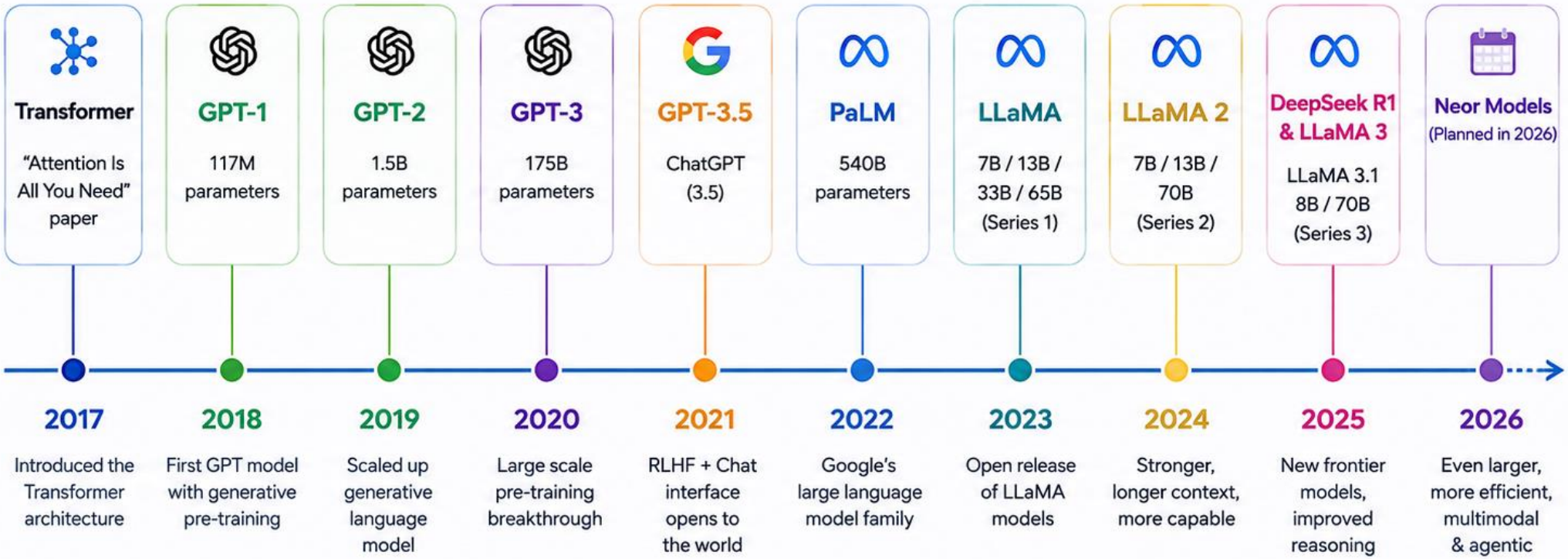
Safety & Ethics
Preventing misuse, ensuring fairness, privacy, and responsible AI.

In Summary

- Huge data is needed
- Powerful hardware is expensive
- Memory and time costs are high
- Training is unstable and complex
- Overfitting and lack of interpretability are real risks
- Building a Transformer from scratch is challenging—but extremely rewarding!

Major Transformer and LLM Releases

2017 to 2026 – Major model and framework milestones



Major Models released in 2026 (expected)

- GPT-5.5
- Gemini 2.5
- Mistral 4
- Claude 4
- LLaMA 4
- Qwen 3
- DeepSeek V3
- Gemiini Ultra 2
- More...

The pace of innovation continues to accelerate every year!

Key Takeaway: From the Transformer paper in 2017 to next-gen models in 2026, the ecosystem has evolved rapidly—bringing larger models, better capabilities, and more accessibility to everyone.

Understand Language Model

What is a Language Model?

- Probability distribution over strings of text
  - how likely is a given string (observation) in a given "language"
  - for example, consider probability for the following four strings
  - English: $p_1 > p_2 > p_3 > p_4$

P_1 = P("a quick brown dog")

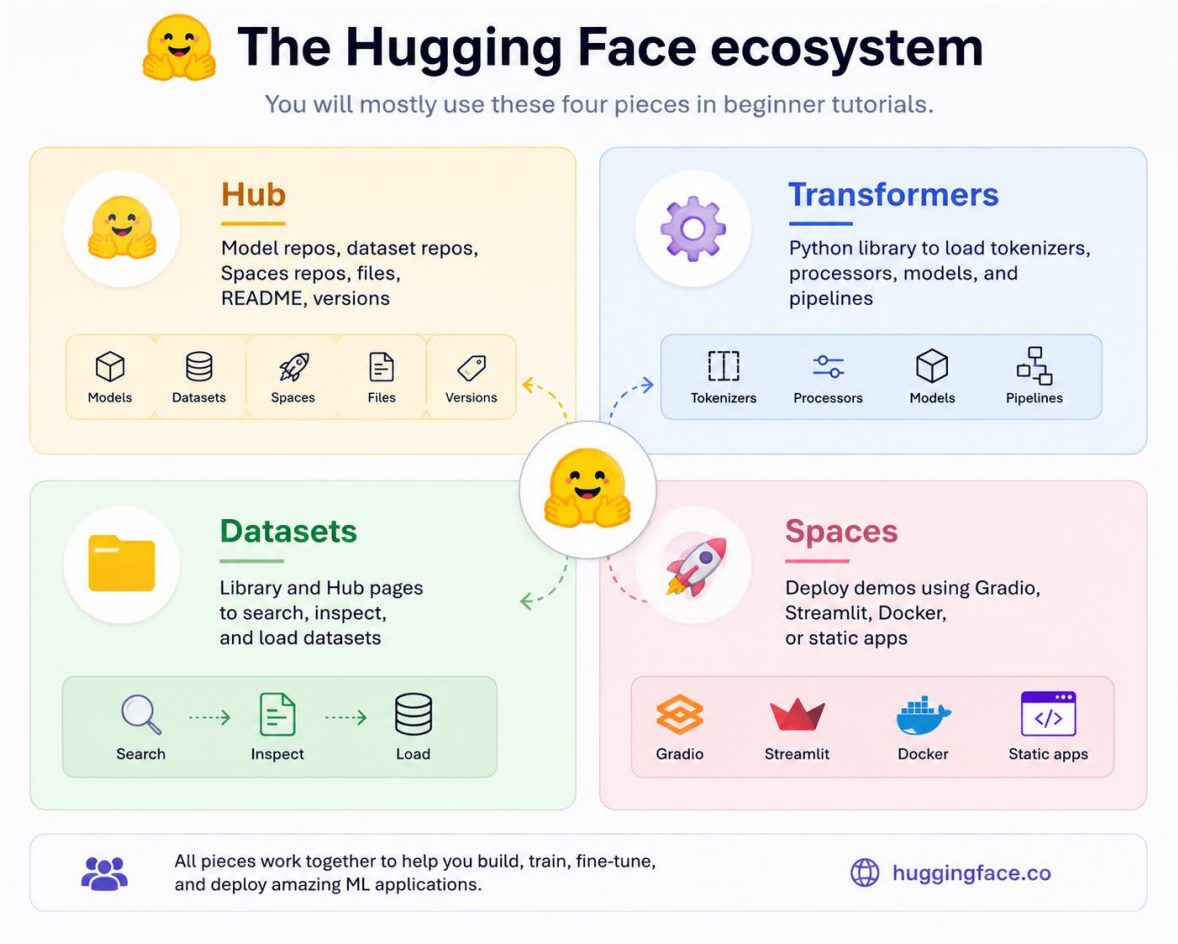
P_2 = P("dog quick a brown")

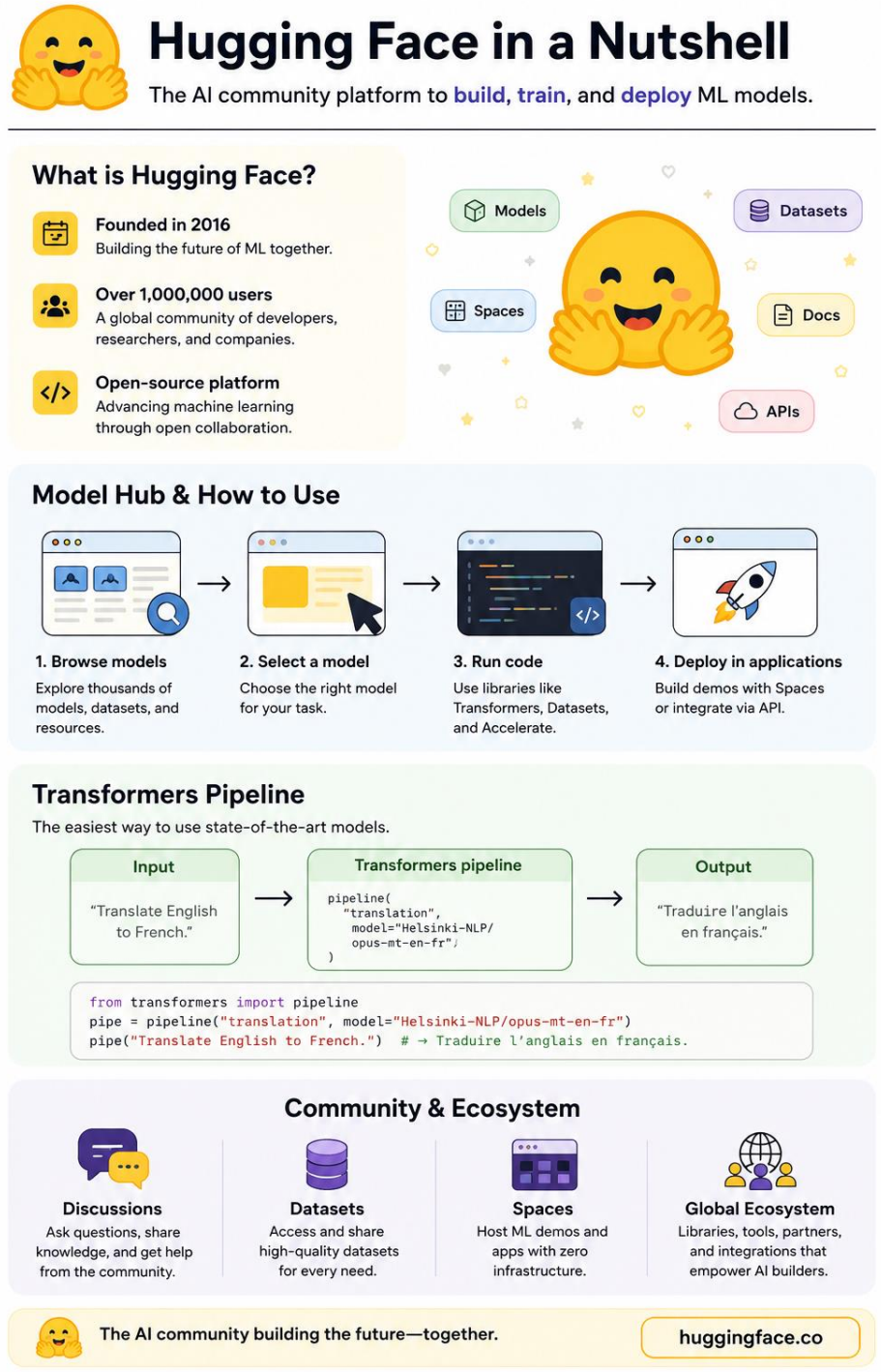
P_3 = P("un chien quick brown")

P_4 = P("un chien brun rapide")

  - … depends on what "language" we are modeling
  - in most of IR we will have $p_1 == p_2$
  - for some applications we will want p_3 to be highly probable

Hugging Face Eco-system



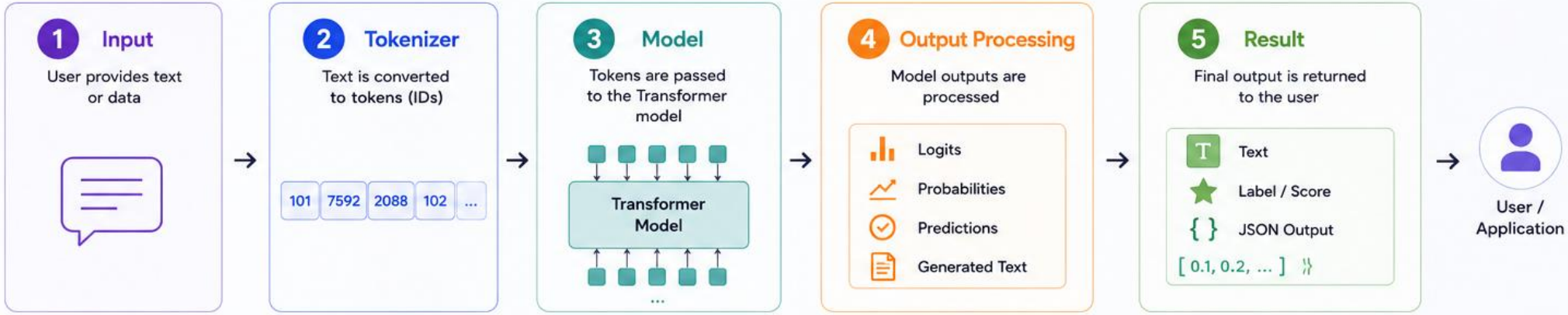


https://huggingface.co/

HF Model Integrations

Transformers Pipeline API – End to End Flow

Simple API abstraction that handles the entire NLP workflow for you!



Inside the Transformers Pipeline API

Preprocessing
- Clean text
- Handle special characters
- Normalize
- ....

Tokenization
- WordPiece / BPE
- Convert to IDs
- Add Special Tokens

101 7592 2088 102

Model Inference
- Pass through Transformer
- Self-Attention
- Feed Forward
- ....

Post-processing
- Decode IDs
- Apply Softmax
- Top-k / Top-p
- Filter Results

Output Formatting
- Text / JSON
- Labels / Scores
- Embeddings
- ....

Common Tasks Using Pipeline API

- Sentiment Analysis
- Text Classification
- Named Entity Recognition (NER)
- Question Answering
- Text Summarization
- Text Generation
- Translation

Example API Call (Python)

```python
from transformers import pipeline

# Create a pipeline for sentiment analysis
classifier = pipeline("sentiment-analysis")

# Use the pipeline
result = classifier("I love using Transformers!")
print(result)
```

Output

```
[
  {
    'label': 'POSITIVE',
    'score': 0.9991
  }
]
```

Why Use Pipeline API?

Easy to Use
One line of code for complex tasks

Saves Time
Handles preprocessing, model inference & post-processing

Reliable
Built on tested and optimized implementations

Flexible
Works for many NLP tasks out of the box

User Input → Transformers Pipeline API → Model Inference → Processed Output → User / Application

Transformers Auto Classes
AutoModel, AutoTokenizer

Hugging Face

⭐ **Auto classes** automatically select the right model architecture and tokenizer based on the model name or path.
They make your code simple, flexible, and future-proof.

AutoTokenizer

Automatically loads the correct tokenizer for any model.

What it does

- Downloads the tokenizer config
- Handles vocabulary
- Tokenizes text into input IDs
- Manages special tokens

Code Example

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

text = "I love NLP!"

encodings = tokenizer(text, return_tensors="pt")

print(encodings)
```

Output

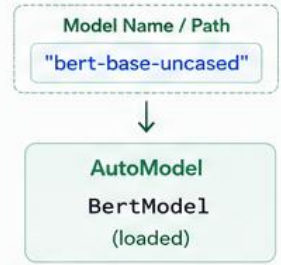
```
{
  'input_ids': tensor([[101, 146, 2293, 999, 102]]),
  'attention_mask': tensor([[1, 1, 1, 1, 1]])
}
```

AutoModel

Automatically loads the correct model architecture (with pre-trained weights).

What it does

- Reads model config
- Detects architecture
- Loads the right model class
- Loads pre-trained weights



Code Example

```
from transformers import AutoModel

model = AutoModel.from_pretrained("bert-base-uncased")

print(model)
```

Output

```
BertModel(
  (embeddings): BertEmbeddings(...
  (encoder): BertEncoder(...
  (pooler): BertPooler(...
)
```

AutoModel for Specific Tasks

Use task-specific AutoModel classes to get outputs directly for your use case.

Common AutoModel classes

AutoModel	Base model (embeddings / hidden states)
AutoModelForSequenceClassification	Text classification
AutoModelForTokenClassification	Token classification / NER
AutoModelForQuestionAnswering	Question answering
AutoModelForCausalLM	Text generation (causal LM)
AutoModelForSeq2SeqLM	Text generation (seq2seq)

Code Example (Text Classification)

```
from transformers import AutoModelForSequenceClassification

model = AutoModelForSequenceClassification.
from_pretrained("distilbert-base-uncased")

outputs = model(**encodings)

print(outputs.logits)
```

Output

```
tensor([[ 2.3456, -1.2345]])
# [positive, negative] scores
```

How They Work Together

AutoTokenizer prepares the input → **AutoModel** processes it → You get the output!

Benefits

- ✓ No need to know the exact model class
- ✓ Works with any model from Hugging Face Hub
- ✓ Less code, more productivity
- ✓ Automatically adapts to future models

Summary

AutoTokenizer	Handles text → tokens
AutoModel	Loads model → gets representations
Task-specific AutoModel*	Gives task-ready outputs

💡 **Tip:** Always use Auto classes unless you have a very specific reason to use a manual class.

🚀 Auto = Simplicity + Flexibility + Power

HF Model Integrations

Auto Classes

Copy page

In many cases, the architecture you want to use can be guessed from the name or the path of the pretrained model you are supplying to the `from_pretrained()` method. AutoClasses are here to do this job for you so that you automatically retrieve the relevant model given the name/path to the pretrained weights/config/vocabulary.

Instantiating one of AutoConfig, AutoModel, and AutoTokenizer will directly create a class of the relevant architecture. For instance

```
model = AutoModel.from_pretrained("google-bert/bert-base-cased", device_map="auto")
```

will create a model that is an instance of BertModel.

There is one class of `AutoModel` for each task.

Extending the Auto Classes

Each of the auto classes has a method to be extended with your custom classes. For instance, if you have defined a custom class of model `NewModel`, make sure you have a `NewModelConfig` then you can add those to the auto classes like this:

```
from transformers import AutoConfig, AutoModel
```

Question - 1

Sentiment Analysis Application

- **Scenario:**
  - You are developing a movie review analysis tool for a streaming platform. The platform wants to automatically classify user reviews as positive or negative using Hugging Face.
- **Question: Using the** pipeline() API, how would you build a sentiment analysis system for the review:
  - "The storyline was amazing, but the ending was disappointing."
- Which Hugging Face pipeline would you use?
- What output do you expect?
- Why is the pipeline() abstraction useful for beginners compared to manually loading models and tokenizers?

Question - 2

Scenario:

A multinational company receives customer feedback in English, Hindi, and Spanish. The default sentiment-analysis model is giving poor results for non-English reviews.

- **Question:**
  You discover the model tabularisai/multilingual-sentiment-analysis on Hugging Face.
- Why would this model be more suitable than the default sentiment analysis pipeline?
- Write the steps required to load and use this model.
- What factors would you consider before deploying it in production (e.g., accuracy, latency, model size, language support)?

Question - 3

Scenario:

You are building an AI-powered educational assistant that generates explanations for engineering concepts. Initially, you used:

pipeline("text-generation", model="Qwen/Qwen2.5-7B-Instruct")

- However, your team now wants:
  - Full control over tokenization,
  - Custom prompt formatting,
  - Access to model inputs/outputs,
  - Optimization for deployment.

Question:

Explain why the team should move from the pipeline() approach to the manual workflow using:

- AutoTokenizer
- AutoModelForCausalLM

- In your answer:
  - Compare the architecture of the Pipeline API and Manual Workflow.
  - Explain the role of tokenization and token IDs in text generation.
  - Describe how the text "Explain Linear Regression" flows through the tokenizer and model before the final output is generated.
  - Discuss the trade-offs between ease of use and flexibility.